Blueprint v0.2 · Refactored

Gridlock Heatmap

From a leaderboard CSV to a traffic-operations platform: forecast, see, report, dispatch.

---

1. Project idea statement

> Gridlock Heatmap turns our GridLock 2.0 demand forecasts into a role-based traffic-operations tool. Operators see where congestion is coming on a time-scrubbable map, log incidents and violations in those zones, and dispatch enforcement. Citizens can see the map and report problems.

The gap today. The pipeline scores about 91.5 to 91.8 out of 100, but its only output is an `Index,demand` CSV. Nobody can act on it.

The bet. Forecasts become useful when they sit next to observed traffic and a workflow for responding to it. The model stays unchanged.

---

2. Project brief

Admin
Manages users and roles, and sees everything.

Operator
Reads forecasts, triages incidents, dispatches officers, closes cases.

Citizen
Views the map and reports incidents, and sees the status of their own reports.

Scope

In (MVP)	Out (for now)
Login with roles (admin, operator, citizen)	Live or streaming inference
Zone heatmap, time slider, hover and click panels	Training or retraining from the UI
Predicted vs observed chart per zone	Real sensor hardware integration

Predicted vs observed chart per zone	Real sensor hardware integration
Incident reporting and status tracking	Cloud deployment, alerting, SSO
Dispatching an officer and recording the resolution	
Batch load of predictions and sensor records	

Key architecture decisions

Decision	Why
Model runs offline in batches; the API never trains or predicts	Flat latency, stateless API, no ML dependencies in the web tier. Batch rows are written with `requested_by` empty.
PostgreSQL, no PostGIS	Cell edges come from the geohash string, so no spatial queries are needed.
FastAPI + SQLAlchemy + Alembic, JWT auth, role checks per endpoint	Typed contracts, versioned migrations, and access rules enforced on the server.
React + Vite, MapLibre, deck.gl	Keyless basemap and GPU-drawn cell polygons that stay smooth during playback.
Zones store a center point only	The frontend and API compute cell polygons from the geohash. One less thing to keep in sync.

Proxy-data assumptions

Dates are proxy: day 48 maps to a configurable anchor date. Several columns have no source in the pipeline data and are derived by one documented script: **junction_name** (from the geohash), **capacity** (from lane count), **vehicle_count**, **predicted_volume** and **average_speed** (from demand), and **congestion_index** (demand scaled to 0 to 100). **model_confidence** has no source at all. **Incidents and dispatches** are synthetic seed data. The real `Weather` column fills `weather_condition`.

Risks and mitigations

Risk	Mitigation
Final predictions are a hand-chained blend of CSVs	An export step joins the chosen file to the test keys, and ingest validates row counts.
Predictions carry only `Index`	Join to `test.csv` to recover geohash and time.
Auth and workflows widen the scope	Build in phases: read-only map first, then login, then incidents and dispatch.
Derived columns could be mistaken	Label proxy data in the UI and keep derivation rules in one

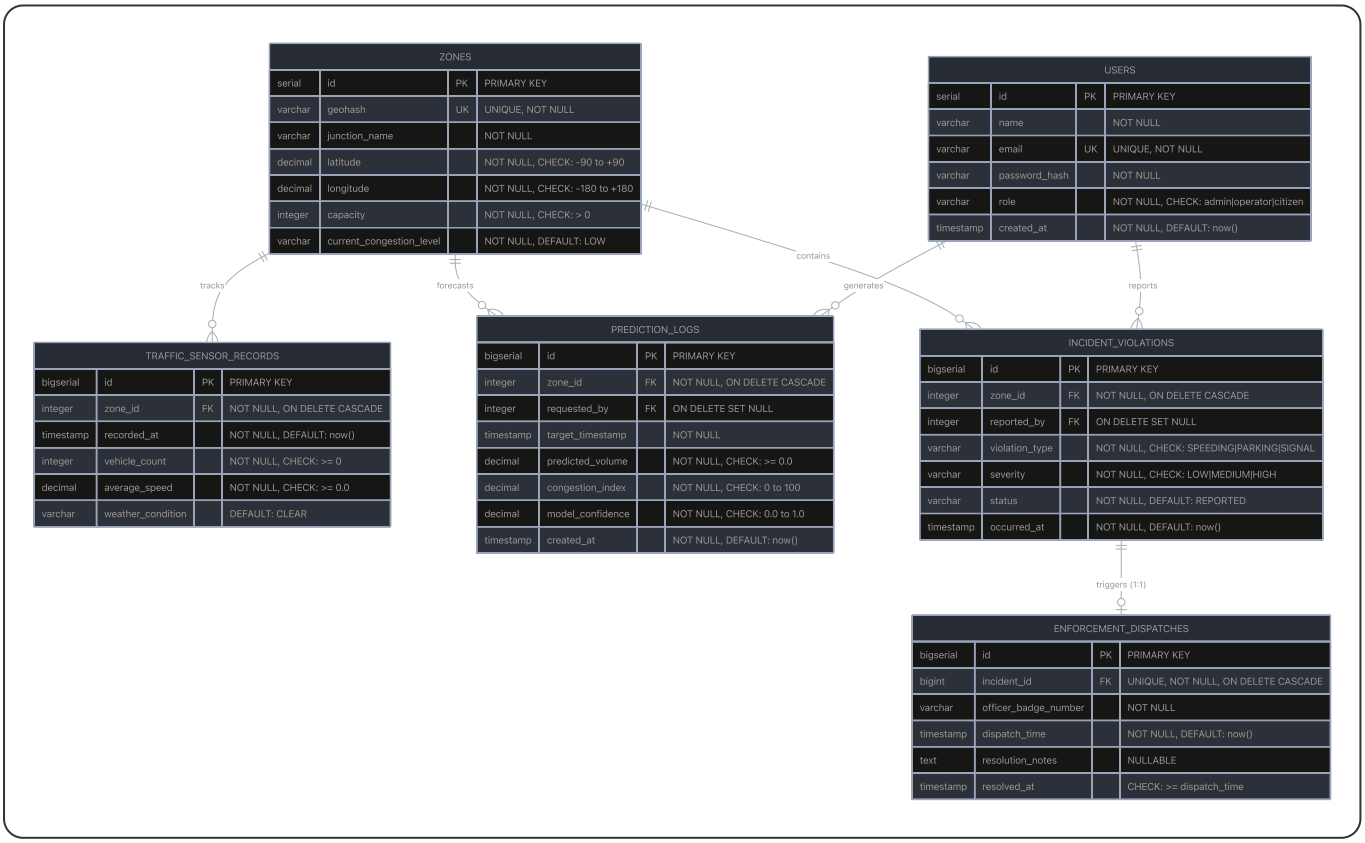
for real data	documented script.

Milestones

Phase	Outcome
M1 · Data	Docker Compose, Postgres, migrations for all six tables, ingest of zones, sensor records and predictions
M2 · API + auth	Login, role checks, zone, heatmap, series and stats endpoints
M3 · Map	Cell heatmap, legend, tooltip, time slider with playback
M4 · Operations	Incident reporting, dispatch and resolution, incident layer and lists
M5 · Polish	Tests, `make dev`, README and demo

3. Entity-relationship diagram

Six tables, six relations, with constraints. Zones are the hub: sensor readings, forecasts and incidents hang off a zone. Full explanation on the ERD page.



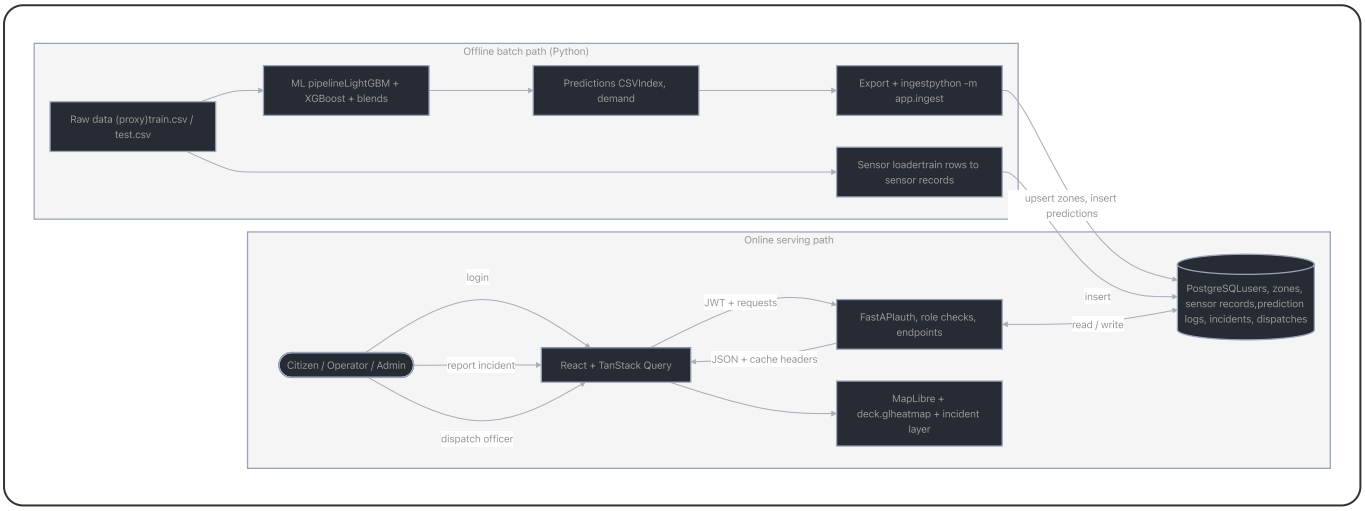
- **Integrity in the database:** CHECK rules, unique keys and delete rules are enforced by Postgres itself, not just the API.

- **History is protected:** deleting a user keeps their reports and requests (SET NULL). Deleting a zone removes its data (CASCADE).
- **One dispatch per incident:** enforced by the unique key on `incident_id`.
- **Open point:** `model_confidence` is NOT NULL but the pipeline has no confidence score. Use a fixed placeholder, or make it nullable (recommended).

---

4. Data flow diagram

The offline path produces data. The online path reads it, and lets users report and dispatch.



---

5. Decision requested